Pontificia Universidad Javeriana
Bases de Datos — Periodo 2610

Designing a Telemetry and UX Database for Chocolate-Doom Research

Samuel Guerra Sánchez
samuel_guerra@javeriana.edu.co

June 2026

1. Introduction

This report documents the design and implementation of a relational database system for storing, managing, and analyzing gameplay telemetry data from a modified version of Chocolate-Doom. The project was developed individually for the Database Systems course (period 2610) at Pontificia Universidad Javeriana.

Chocolate-Doom is a source port of the classic 1993 first-person shooter DOOM by id Software. The modified build used in this project emits per-tic telemetry data — position coordinates (x, y, z), facing angle, momentum vector, field of view (FOV), and combat statistics (health, armor, ammo) — at a rate of 35 tics per second. Additionally, the system integrates user experience (UX) survey data collected through the BANGS (Basic Needs in Games Scale) instrument, enabling cross-analysis between gameplay behavior and psychological need satisfaction.

The primary goal of this project is to design a normalized relational schema in PostgreSQL that supports exploratory and analytical queries over trajectory, proximity, and cooperation patterns, while enforcing data quality, referential integrity, and research ethics compliance.

1.1 Project Objectives

1. Design a conceptual ER model capturing all relevant entities, relationships, and cardinalities.
2. Derive a normalized relational schema (3NF) with complete constraints.
3. Implement the schema in PostgreSQL with DDL, integrity rules, and sample data.
4. Design an ETL pipeline for ingesting raw TSV telemetry logs.
5. Implement and evaluate analytical queries, indexes, and views.
6. Address privacy, consent, and research ethics through schema design and documentation.

2. Project Development

2.1 Domain Assumptions and Requirements

The following assumptions were established to resolve ambiguities in the project guidelines and guide all subsequent design decisions:

#	Assumption
1	User-Player relationship is 1:N — one user may have multiple player identities.
2	Player-Game relationship is N:M — a player may participate in many games and a game may have many players.
3	Game-Map relationship is 1:1 — each game session occurs on exactly one map.
4	Users may revoke consent at any time; associated records are anonymized (user_id set to NULL), not deleted.
5	Player proximity is defined as Euclidean distance ≤ 300 map units.
6	BANGS (Basic Needs in Games Scale) is used as the UX instrument.
7	Tics are numbered starting from 1.
8	Health range: 0–200; Armor range: 0–200; Ammo: >= 0 with no upper limit.
9	Kills and assists are session-level aggregated stats stored in GameParticipant.
10	Maximum 8 players per game session.
11	Proximity distance is calculated between consecutive tics (min: 2, max: 35 tics).

2.2 Conceptual ER Model

The entity-relationship diagram captures twelve entities organized into two main branches: the telemetry branch and the UX survey branch. Both branches are connected through the User entity.

The telemetry branch follows this hierarchy: Episode → Map → Sector, with Game linking to Map and TelemetryEvent recording per-tic state for each player in each game. The N:M relationship between Player and Game is resolved through the GameParticipant bridge table.

The UX branch connects User to UXResponse, which links to individual UXResponseItem records. Each item corresponds to a specific UXItem in a UXInstrument definition.

Relationship	Cardinality	Description
User → Player	1:N	One user may have multiple player identities
Player ↔ Game	N:M via GameParticipant	Players participate in multiple games
Game → Map	N:1	Each game occurs on one map

Episode → Map	1:N	Each episode contains multiple maps
Map → Sector	1:N	Each map is divided into sectors
Game → TelemetryEvent	1:N	Each game generates many telemetry records
Player → TelemetryEvent	1:N	Each player generates many telemetry records
TelemetryEvent → Sector	N:1	Many events occur in the same sector
User → UXResponse	1:N	One user completes multiple surveys
UXInstrument → UXItem	1:N	Each instrument has multiple items
UXResponse ↔ UXItem	N:M via UXResponseItem	Each response covers all instrument items

2.3 Relational Schema and Data Dictionary

The relational schema consists of twelve tables derived from the ER model. All tables are normalized to Third Normal Form (3NF). The following table summarizes the schema:

Table	PK	FKs	Key Constraints
Episode	episode_id (SERIAL)	—	episode_name NOT NULL
Map	map_id (SERIAL)	episode_id → Episode	map_code NOT NULL
Sector	sector_id (SERIAL)	map_id → Map	min_x, max_x, min_y, max_y NOT NULL
UXInstrument	instrument_id (SERIAL)	—	instrument_name NOT NULL
UXItem	item_id (SERIAL)	instrument_id → UXInstrument	subscale NOT NULL
User	user_id (SERIAL)	—	age CHECK > 0, consent DEFAULT TRUE
Player	player_id (SERIAL)	user_id → User	nickname NOT NULL
Game	game_id (SERIAL)	map_id → Map	start_ts NOT NULL
GameParticipant	(player_id, game_id)	player_id, game_id	kills/assists CHECK >= 0
UXResponse	response_id (SERIAL)	user_id (nullable), instrument_id, game_id	response_date DEFAULT NOW()

UXResponseItem	(response_id, item_id)	response_id, item_id	value CHECK BETWEEN 1 AND 5
TelemetryEvent	event_id (SERIAL)	game_id, player_id, sector_id	UNIQUE(game_id, player_id, tic)

3NF Justification

All tables satisfy First Normal Form (no repeating groups), Second Normal Form (all non-key attributes depend on the full primary key), and Third Normal Form (no transitive dependencies between non-key attributes). The only intentional denormalization is the optional caching of episode_id in TelemetryEvent for query performance, which is documented as a design decision.

2.4 DDL Implementation

The complete DDL is implemented in sql/01_schema.sql. Below is the critical TelemetryEvent table definition as a representative example:

```
CREATE TABLE TelemetryEvent (
    event_id   SERIAL PRIMARY KEY,
    game_id    INT NOT NULL REFERENCES Game(game_id),
    player_id  INT NOT NULL REFERENCES Player(player_id),
    sector_id  INT REFERENCES Sector(sector_id),
    tic        INT NOT NULL CHECK (tic >= 1),
    pos_x      FLOAT8 NOT NULL,
    pos_y      FLOAT8 NOT NULL,
    pos_z      FLOAT8 NOT NULL,
    health     INT CHECK (health >= 0),
    armor      INT CHECK (armor BETWEEN 0 AND 200),
    ammo       INT CHECK (ammo >= 0),
    UNIQUE (game_id, player_id, tic)
);
```

2.5 ETL Pipeline

The ETL pipeline follows a three-stage process: Extract, Transform, Load. The modified Chocolate-Doom engine emits telemetry as tab-separated values (TSV) files. The pipeline is implemented in sql/05_etl.sql and consists of:

7. Extract: Raw TSV data is loaded into a staging table (stg_telemetry) with all fields as TEXT, using COPY with no constraints.
8. Transform & Load: An INSERT...SELECT statement validates field formats using regex patterns, casts to proper types, and inserts into TelemetryEvent. Duplicate records (same game_id, player_id, tic) are silently ignored via ON CONFLICT DO NOTHING.
9. Error Logging: Malformed records are captured in etl_error_log for audit purposes.

Synthetic data was generated using the Python script etl/generate_data.py, producing 51,000 telemetry rows across 3 episodes, 9 maps, 18 games, and 8 players. All records passed validation (0 malformed records).

2.6 Analytical Queries

Six analytical queries were implemented in sql/04_queries.sql. All queries were executed against the populated database and returned expected results:

Query	Description	Result
Q1	Average duration of game sessions per map	9 maps, avg 2250 sec each
Q3	Shortest and longest trajectory distances per player	8 players, distances ranging ~5800–6555 units
Q4	UX responses for players with above-average trajectory duration	36 rows returned
Q5	Most visited sector (hotspot) per episode and map	54 sector-map combinations ranked by visits
Q6	Number of tics where players were together in a sector	Co-presence detected across all 18 games
Q8	Total distance traveled and average speed per player	DemonBane led with ~94,153 total units

Query 3 — Window Function Example

Query 3 demonstrates the use of window functions (LAG) to calculate step-by-step Euclidean distances between consecutive tics:

```
WITH steps AS (
    SELECT player_id, game_id,
           SQRT(POWER(pos_x - LAG(pos_x) OVER w, 2) +
                POWER(pos_y - LAG(pos_y) OVER w, 2)) AS step_dist
    FROM TelemetryEvent
    WINDOW w AS (PARTITION BY game_id, player_id ORDER BY tic)
)
SELECT player_id, MIN(total_dist), MAX(total_dist)
FROM totals GROUP BY player_id;
```

2.7 Index Evaluation

Three indexes were created and evaluated using EXPLAIN (ANALYZE, BUFFERS). The results demonstrate significant performance improvements for queries on large tables:

Index	Table	Columns	Without Index	With Index	Speedup
idx_tel_game_player_tic	TelemetryEvent	game_id, player_id, tic	Seq Scan — 50.620 ms	Index Scan — 0.164 ms	~309x
idx_tel_sector	TelemetryEvent	sector_id, game_id	Seq Scan — 17.445 ms	Bitmap Index Scan — 0.249 ms	~70x
idx_gp_player_game	GameParticipant	player_id, game_id	Seq Scan — 0.047 ms	Seq Scan — 0.042 ms	Minimal*

* The idx_gp_player_game index showed minimal improvement because GameParticipant only contains 20 rows in the test dataset. PostgreSQL's query planner correctly opts for a sequential scan on small tables. The index will provide significant benefit as the table grows to thousands of rows in production.

2.8 Views and Materialized View

Two regular views and one materialized view were created in sql/03_views.sql to support frequent analytical use cases:

View	Type	Purpose
v_trajectories	Regular View	Returns full ordered trajectory per player per game (game_id, player_id, tic, pos_x, pos_y, pos_z, sector_id)
v_ux_summary	Regular View	Returns average BANGS score per user per subscale (Autonomy, Competence, Relatedness)
mv_player_traj_stats	Materialized View	Pre-computes total distance, total tics, and average step distance per player per game — avoids expensive recalculation on large datasets

The materialized view is refreshed with REFRESH MATERIALIZED VIEW mv_player_traj_stats whenever new telemetry data is loaded into the database.

2.9 Ethics and Privacy

The project was developed in compliance with Colombian data protection legislation, specifically Law 1581 of 2012, which establishes general provisions for the protection of personal data (Habeas Data).

Key privacy provisions implemented in the schema and process:

- Explicit consent is required before storing any user data — the consent field in the User table must be TRUE.
- Structural separation between identifiable data (User table) and telemetry data (TelemetryEvent) — they are linked only through Player.
- Consent revocation anonymizes rather than deletes — user_id is set to NULL in UXResponse and Player is unlinked, preserving research data integrity.

- Data access is restricted to authorized users only (system administrators and the data subject).
- Users are responsible for the accuracy and authenticity of their personal data.

3. Conclusions

This project successfully demonstrates the full lifecycle of a relational database system applied to a real-world research domain: from conceptual modeling to physical implementation, ETL ingestion, analytical querying, and performance evaluation.

The resulting schema in PostgreSQL provides a solid foundation for analyzing gameplay telemetry from Chocolate-Doom sessions. The 3NF-normalized design ensures data integrity while allowing flexible querying across telemetry, spatial, and UX dimensions. The ETL pipeline reliably ingests 51,000+ rows from raw TSV logs with full validation and error handling.

Index evaluation confirmed that strategic indexing on TelemetryEvent yields dramatic performance improvements — up to 309x faster query execution — which is critical given that the table grows at 35 tics/second per player. The three indexes created target the most frequent query patterns: filtering by game and player, filtering by sector, and joining through GameParticipant.

The integration of BANGS survey data with telemetry enables meaningful cross-domain analysis, such as correlating movement patterns with psychological need satisfaction scores. This cross-analysis is demonstrated in Query 4 and the v_ux_summary view.

From a data ethics perspective, the schema enforces consent management and data anonymization at the database level rather than relying solely on application logic, ensuring compliance with Colombian Law 1581 of 2012.

Future work could include enabling the PostGIS extension for true spatial indexing on player positions, implementing real-time proximity detection using triggers, and expanding the UX instrument coverage to include PENS or GUESS for comparative analysis.

Appendix: Project Repository

All source code, SQL scripts, and documentation are available in the public GitHub repository:

`https://github.com/samuguerraa/doom`

File	Description
sql/01_schema.sql	Complete DDL — all CREATE TABLE statements
sql/02_indexes.sql	Index definitions
sql/03_views.sql	Regular and materialized views
sql/04_queries.sql	Six analytical queries
sql/05_etl.sql	ETL pipeline — staging to core
etl/generate_data.py	Python synthetic data generator
docs/er_diagram.png	Entity-Relationship diagram
docs/schema.md	Relational schema documentation
docs/Data_dictionary.pdf	Complete data dictionary
docs/requirements.md	Domain assumptions, requirements, and ethics
setup.sh	Single-script database recreation